

AQL en ArangoDB: consultas simples, intermedias y avanzadas

Documentos, agregaciones, subconsultas, grafos y rendimiento

Alex Di Genova

Curso de Bases de Datos / Minería de Datos

2 de julio de 2026

Objetivos de aprendizaje

- Comprender el rol de AQL como lenguaje declarativo de consulta y manipulación de datos en ArangoDB.
- Escribir consultas básicas con FOR, FILTER, SORT, LIMIT, LET y RETURN.
- Manipular documentos con INSERT, UPDATE, REPLACE, REMOVE y UPSERT.
- Construir consultas intermedias con COLLECT, subconsultas, proyecciones y parámetros.
- Aplicar consultas avanzadas sobre grafos, rutas, optimización y buenas prácticas.

¿Qué es AQL?

AQL significa **ArangoDB Query Language**. Es el lenguaje usado por ArangoDB para consultar y modificar datos almacenados como documentos JSON, colecciones y grafos.

Idea central

AQL describe **qué resultado se quiere obtener**, no necesariamente **cómo** obtenerlo. Por eso se considera un lenguaje declarativo.

- Similar en propósito a SQL, pero con sintaxis adaptada a documentos y grafos.
- Es un lenguaje de manipulación de datos: permite leer, insertar, actualizar y eliminar documentos.
- No se usa para crear bases, colecciones o índices; esas tareas son administrativas.

Modelo mental: de SQL a AQL

Concepto	SQL	AQL
Tabla	estudiantes	Colección estudiantes
Fila	Registro	Documento JSON
Columna	Atributo fijo	Atributo flexible
WHERE	WHERE	FILTER
SELECT	SELECT	RETURN
GROUP BY	GROUP BY	COLLECT
JOIN	JOIN	bucles, DOCUMENT(), grafos
Relación	FK	documento, edge o referencia

Diferencia clave

En AQL se trabaja naturalmente con objetos, arreglos y grafos, no solo con tablas planas.

Caso de estudio

Usaremos una base de datos pequeña de cursos universitarios:

- **estudiantes:** documentos con nombre, carrera, edad, promedio e intereses.
- **cursos:** documentos con nombre, área y créditos.
- **inscripciones:** edge collection desde estudiante hacia curso, con nota y semestre.
- **sigue:** edge collection estudiante-estudiante, para consultas de red social.
- **requiere:** edge collection curso-curso, para prerequisites.

Idea

El mismo conjunto de datos permite practicar consultas de documentos, agregaciones y grafos.

Preparación: colecciones fuera de AQL

AQL no crea colecciones. Primero se preparan desde la interfaz web, arangosh o un driver.

```
// En arangosh, no en AQL puro:  
db._create("estudiantes")  
db._create("cursos")  
db._createEdgeCollection("inscripciones")  
db._createEdgeCollection("sigue")  
db._createEdgeCollection("requiere")
```

Importante

Las consultas AQL asumen que las colecciones ya existen.

Datos de ejemplo: estudiantes

```
FOR doc IN [  
  { _key:"e1", nombre:"Ana", edad:21,  
    carrera:"Bioinformatica", promedio:6.1,  
    activo:true, intereses:["genomica", "R", "grafos"] },  
  { _key:"e2", nombre:"Bruno", edad:23,  
    carrera:"Ingenieria", promedio:5.4,  
    activo:true, intereses:["bases de datos", "Python"] },  
  { _key:"e3", nombre:"Carla", edad:20,  
    carrera:"Bioinformatica", promedio:6.5,  
    activo:true, intereses:["IA", "histologia"] },  
  { _key:"e4", nombre:"Diego", edad:24,  
    carrera:"Medicina", promedio:4.9,  
    activo:false, intereses:["salud", "estadistica"] }  
]  
INSERT doc INTO estudiantes
```

Datos de ejemplo: cursos

```
FOR doc IN [
  { _key:"bd", nombre:"Bases de Datos", area:"datos", credits:5 },
  { _key:"prog", nombre:"Programacion", area:"programacion", credits:6 },
  { _key:"mineria", nombre:"Mineria de Datos", area:"datos", credits:5 },
  { _key:"bioinfo", nombre:"Bioinformatica", area:"biologia", credits:4 }
]
INSERT doc INTO cursos
```

Observación

Cada documento tiene una llave `_key`; ArangoDB también genera `_id` y `_rev`.

Datos de ejemplo: relaciones

```
FOR doc IN [
  { _from:"estudiantes/e1", _to:"cursos/bd", nota:6.4, semestre:"2026-1" },
  { _from:"estudiantes/e1", _to:"cursos/mineria", nota:6.0, semestre:"2026-1" },
  { _from:"estudiantes/e2", _to:"cursos/bd", nota:5.3, semestre:"2026-1" },
  { _from:"estudiantes/e3", _to:"cursos/mineria", nota:6.7, semestre:"2026-1" },
  { _from:"estudiantes/e4", _to:"cursos/prog", nota:4.5, semestre:"2026-1" }
]
INSERT doc INTO inscripciones

FOR doc IN [
  { _from:"estudiantes/e1", _to:"estudiantes/e2" },
  { _from:"estudiantes/e2", _to:"estudiantes/e3" },
  { _from:"estudiantes/e3", _to:"estudiantes/e4" }
]
INSERT doc INTO sigue
```

Datos de ejemplo: prerrequisitos

```
FOR doc IN [  
  { _from:"cursos/mineria", _to:"cursos/bd" },  
  { _from:"cursos/mineria", _to:"cursos/prog" },  
  { _from:"cursos/bioinfo", _to:"cursos/prog" }  
]  
INSERT doc INTO requiere
```

Interpretación

Si existe una arista cursos/mineria -> cursos/bd, entonces Minería de Datos requiere Bases de Datos.

1. Primer resultado con RETURN

Toda consulta de lectura debe producir un resultado con RETURN.

```
RETURN "Hola AQL"
```

Resultado esperado:

```
[ "Hola AQL" ]
```

Clave

El resultado de una consulta AQL siempre es un arreglo, incluso si se devuelve un solo valor.

2. Recorrer una colección con FOR

```
FOR e IN estudiantes  
  RETURN e
```

- FOR e IN estudiantes: itera por cada documento de la colección.
- RETURN e: devuelve el documento completo.

Cuidado

En colecciones grandes, devolver documentos completos puede ser costoso. Conviene proyectar solo los atributos necesarios.

3. Proyecciones: devolver solo lo necesario

```
FOR e IN estudiantes
  RETURN {
    nombre: e.nombre,
    carrera: e.carrera,
    promedio: e.promedio
  }
```

Resultado conceptual

Una lista de objetos más pequeños, no el documento completo.

```
[
  { "nombre": "Ana", "carrera": "Bioinformatica", "promedio": 6.1 },
  { "nombre": "Bruno", "carrera": "Ingenieria", "promedio": 5.4 }
]
```

4. Filtros básicos

```
FOR e IN estudiantes  
  FILTER e.carrera == "Bioinformatica"  
  RETURN e.nombre
```

Operadores comunes:

- Comparación: ==, !=, <, <=, >, >=
- Pertenencia: IN, NOT IN
- Texto: LIKE, =~ para expresiones regulares
- Lógicos: AND, OR, NOT

5. Filtros compuestos

```
FOR e IN estudiantes
  FILTER e.activo == true
  FILTER e.promedio >= 6.0
  RETURN { nombre: e.nombre, promedio: e.promedio }
```

Equivalente:

```
FOR e IN estudiantes
  FILTER e.activo == true AND e.promedio >= 6.0
  RETURN { nombre: e.nombre, promedio: e.promedio }
```

Buena práctica

Usar varios FILTER puede mejorar la legibilidad. El optimizador decide cómo ejecutar la consulta.

6. Ordenar y limitar

```
FOR e IN estudiantes
  FILTER e.activo == true
  SORT e.promedio DESC, e.nombre ASC
  LIMIT 3
  RETURN { nombre: e.nombre, promedio: e.promedio }
```

Orden importa

Usualmente se filtra, luego se ordena y al final se limita. Un LIMIT antes de FILTER puede descartar documentos relevantes.

7. Paginación

```
FOR e IN estudiantes  
  SORT e.nombre ASC  
  LIMIT 0, 10  
  RETURN e.nombre
```

```
FOR e IN estudiantes  
  SORT e.nombre ASC  
  LIMIT 10, 10  
  RETURN e.nombre
```

- LIMIT 0, 10: primera página.
- LIMIT 10, 10: segunda página.

8. Variables temporales con LET

```
FOR e IN estudiantes
  LET estado = e.promedio >= 5.5 ? "aprobado" : "riesgo"
  RETURN {
    nombre: e.nombre,
    promedio: e.promedio,
    estado: estado
  }
```

Uso de LET

Permite guardar cálculos intermedios, mejorar legibilidad y reutilizar expresiones.

9. Parámetros enlazados

Los parámetros evitan concatenar texto en las consultas y ayudan a escribir código más seguro.

```
FOR e IN estudiantes
  FILTER e.carrera == @carrera
  FILTER e.promedio >= @min_promedio
  RETURN e
```

Parámetros:

```
{
  "carrera": "Bioinformatica",
  "min_promedio": 6.0
}
```

10. Parámetro para nombre de colección

```
FOR doc IN @@coleccion  
  LIMIT 5  
  RETURN doc
```

Parámetros:

```
{  
  "@coleccion": "estudiantes"  
}
```

Nota

Para valores se usa @param; para colecciones se usa @@param en la consulta y "@param" en el JSON de parámetros.

11. Arreglos: expansión

```
FOR e IN estudiantes
  RETURN {
    nombre: e.nombre,
    intereses: e.intereses,
    primer_interes: e.intereses[0]
  }
```

Extraer todos los intereses en una lista plana:

```
RETURN (
  FOR e IN estudiantes
    RETURN e.intereses
)[**]
```

12. Filtrar dentro de arreglos

```
FOR e IN estudiantes
  RETURN {
    nombre: e.nombre,
    intereses_datos: e.intereses[* FILTER
      CONTAINS(LOWER(CURRENT), "datos") OR
      CONTAINS(LOWER(CURRENT), "grafos")
    ]
  }
```

CURRENT

Dentro de una expresión inline sobre arreglos, CURRENT representa el elemento actual.

13. DOCUMENT(): recuperar por identificador

```
RETURN DOCUMENT("estudiantes/e1")
```

Usando una arista de inscripción:

```
FOR i IN inscripciones
  LET estudiante = DOCUMENT(i._from)
  LET curso = DOCUMENT(i._to)
  RETURN {
    estudiante: estudiante.nombre,
    curso: curso.nombre,
    nota: i.nota
  }
```

14. Patrón de join con DOCUMENT()

Consulta: estudiantes inscritos en Bases de Datos.

```
FOR i IN inscripciones
  FILTER i._to == "cursos/bd"
  LET e = DOCUMENT(i._from)
  RETURN {
    estudiante: e.nombre,
    carrera: e.carrera,
    nota: i.nota
  }
```

Lectura

La arista tiene `_from` y `_to`; `DOCUMENT()` recupera los documentos conectados.

15. Patrón de join con bucles

```
FOR e IN estudiantes
  FOR i IN inscripciones
    FILTER i._from == e._id
    LET c = DOCUMENT(i._to)
    RETURN {
      estudiante: e.nombre,
      curso: c.nombre,
      nota: i.nota
    }
```

Rendimiento

Este patrón puede ser costoso si no hay índices adecuados. Para relaciones naturales, una edge collection y traversal pueden ser mejores.

16. Agrupar con COLLECT

Contar estudiantes por carrera:

```
FOR e IN estudiantes  
  COLLECT carrera = e.carrera WITH COUNT INTO n  
  SORT n DESC  
  RETURN { carrera: carrera, estudiantes: n }
```

Equivalente conceptual

COLLECT cumple el rol de GROUP BY, pero puede agrupar documentos, calcular agregados y construir arreglos.

17. Agregaciones por grupo

Promedio de notas por curso:

```
FOR i IN inscripciones
  LET c = DOCUMENT(i._to)
  COLLECT curso = c.nombre
  AGGREGATE
    n = COUNT(),
    promedio = AVERAGE(i.nota),
    maxima = MAX(i.nota)
  SORT promedio DESC
  RETURN {
    curso: curso,
    n: n,
    promedio: ROUND(promedio * 100) / 100,
    maxima: maxima
  }
```

18. COLLECT INTO: conservar miembros del grupo

```
FOR i IN inscripciones
  LET e = DOCUMENT(i._from)
  LET c = DOCUMENT(i._to)
  COLLECT curso = c.nombre INTO grupo = {
    estudiante: e.nombre,
    nota: i.nota
  }
RETURN {
  curso: curso,
  inscritos: grupo[*].estudiante,
  notas: grupo[*].nota
}
```

Cuándo usarlo

Cuando no solo necesitamos estadísticas, sino también los elementos que pertenecen a cada grupo.

19. RETURN DISTINCT vs COLLECT

Valores únicos simples:

```
FOR e IN estudiantes  
  RETURN DISTINCT e.carrera
```

Agrupación flexible:

```
FOR e IN estudiantes  
  COLLECT carrera = e.carrera  
  RETURN carrera
```

- RETURN DISTINCT: simple, para un solo valor.
- COLLECT: más flexible; permite contar, agregar y agrupar por varias variables.

20. Subconsultas con LET

Para cada estudiante, listar sus cursos:

```
FOR e IN estudiantes
  LET cursos_tomados = (
    FOR i IN inscripciones
      FILTER i._from == e._id
      LET c = DOCUMENT(i._to)
      SORT c.nombre
      RETURN { curso: c.nombre, nota: i.nota }
  )
RETURN {
  estudiante: e.nombre,
  n_cursos: LENGTH(cursos_tomados),
  cursos: cursos_tomados
}
```

21. Subconsultas: top-k por estudiante

Mejores notas de cada estudiante:

```
FOR e IN estudiantes
  LET mejores = (
    FOR i IN inscripciones
      FILTER i._from == e._id
      LET c = DOCUMENT(i._to)
      SORT i.nota DESC
      LIMIT 2
      RETURN { curso: c.nombre, nota: i.nota }
  )
  RETURN { estudiante: e.nombre, mejores: mejores }
```

Idea

Las subconsultas permiten resolver problemas jerárquicos: un documento principal con resultados anidados.

22. INSERT y RETURN NEW

```
INSERT {  
  _key: "e5",  
  nombre: "Elena",  
  edad: 22,  
  carrera: "Ingenieria",  
  promedio: 5.8,  
  activo: true  
} INTO estudiantes  
RETURN NEW
```

NEW

En consultas de modificación, NEW representa el documento insertado o modificado.

23. UPDATE, REPLACE y REMOVE

Actualización parcial:

```
UPDATE "e5" WITH { promedio: 6.0 } IN estudiantes  
RETURN NEW
```

Reemplazo completo:

```
REPLACE "e5" WITH {  
  nombre:"Elena", edad:22, carrera:"Ingenieria", activo:true  
} IN estudiantes  
RETURN NEW
```

Eliminar:

```
REMOVE "e5" IN estudiantes  
RETURN OLD
```

24. UPDATE masivo

Marcar estudiantes inactivos con bajo promedio:

```
FOR e IN estudiantes
  FILTER e.promedio < 5.0
  UPDATE e WITH { activo: false } IN estudiantes
  RETURN {
    antes: OLD.activo,
    despues: NEW.activo,
    estudiante: NEW.nombre
  }
```

Precaución

Siempre pruebe primero la parte FOR + FILTER + RETURN antes de ejecutar un UPDATE o REMOVE masivo.

25. UPSERT

Insertar si no existe; actualizar si ya existe:

```
UPSERT { _key: "e1" }  
  INSERT {  
    _key: "e1", nombre:"Ana", visitas:1  
  }  
  UPDATE {  
    visitas: OLD.visitas ? OLD.visitas + 1 : 1  
  }  
IN estudiantes  
LET tipo = IS_NULL(OLD) ? "insert" : "update"  
RETURN { operacion: tipo, estudiante: NEW.nombre, visitas: NEW.visitas }
```

Uso típico

Contadores, estados acumulados, sincronización incremental de datos.

Grafos en AQL

- Un grafo tiene **vértices**: documentos normales.
- Y **aristas**: documentos especiales con `_from` y `_to`.
- AQL permite recorrer grafos con `OUTBOUND`, `INBOUND` o `ANY`.

Sintaxis general

```
FOR vertice, arista, camino IN min..max DIRECCION inicio edgeCollection  
RETURN ...
```

- `vertice`: nodo visitado.
- `arista`: arista usada para llegar al nodo.
- `camino`: ruta completa con `vertices` y `edges`.

26. Traversal simple: red social

Contactos hasta distancia 2 desde Ana:

```
FOR v, e, p IN 1..2 OUTBOUND "estudiantes/e1" sigue
  RETURN {
    contacto: v.nombre,
    profundidad: LENGTH(p.edges),
    camino: p.vertices[*].nombre
  }
```

Interpretación

1..2 significa seguir caminos de mínimo 1 y máximo 2 aristas.

27. Traversal con filtros

Contactos activos de Ana hasta distancia 2:

```
FOR v, e, p IN 1..2 OUTBOUND "estudiantes/e1" sigue
  FILTER v.activo == true
  SORT LENGTH(p.edges), v.nombre
  RETURN {
    contacto: v.nombre,
    carrera: v.carrera,
    distancia: LENGTH(p.edges)
  }
```

Cuidado

El filtro se aplica a los vértices emitidos. Si se desea cortar ramas de búsqueda, se usa PRUNE.

28. PRUNE: cortar exploración

Prerrequisitos de Minería de Datos, cortando cuando se llega a Programación:

```
FOR v, e, p IN 1..3 OUTBOUND "cursos/mineria" requiere
  PRUNE v._key == "prog"
  RETURN {
    prerrequisito: v.nombre,
    camino: p.vertices[*].nombre
  }
```

Diferencia

FILTER decide qué resultados se devuelven. PRUNE decide qué ramas del grafo se siguen explorando.

29. Opciones de traversal

```
FOR v, e, p IN 1..3 OUTBOUND "estudiantes/e1" sigue
  OPTIONS {
    order: "bfs",
    uniqueVertices: "path"
  }
RETURN p.vertices[*].nombre
```

- order: "bfs": explora por amplitud.
- uniqueVertices: "path": evita repetir vértices dentro del mismo camino.
- Útil para controlar ciclos y el tamaño de la exploración.

30. Camino más corto

```
FOR v, e IN OUTBOUND_SHORTEST_PATH  
  "estudiantes/e1" TO "estudiantes/e4" sigue  
RETURN v.nombre
```

Pregunta que responde

¿Cuál es la secuencia mínima de relaciones sigue para llegar desde Ana hasta Diego?

Nota

Para grafos con pesos, se pueden usar opciones de peso en variantes de caminos.

31. Recomendación simple usando grafos

Recomendar contactos de segundo grado que Ana aún no sigue directamente:

```
LET directos = (  
  FOR v IN 1..1 OUTBOUND "estudiantes/e1" sigue  
    RETURN v._id  
)  
  
FOR v, e, p IN 2..2 OUTBOUND "estudiantes/e1" sigue  
  FILTER v._id NOT IN directos  
  FILTER v._id != "estudiantes/e1"  
  RETURN DISTINCT {  
    recomendado: v.nombre,  
    via: p.vertices[1].nombre  
  }
```

32. Consulta mixta: documentos + grafos

Para cada estudiante activo, recuperar cursos y contactos de primer grado:

```
FOR e IN estudiantes
  FILTER e.activo
  LET cursos = (
    FOR i IN inscripciones
      FILTER i._from == e._id
      LET c = DOCUMENT(i._to)
      RETURN c.nombre
  )
  LET contactos = (
    FOR v IN 1..1 OUTBOUND e._id sigue
      RETURN v.nombre
  )
  RETURN { estudiante: e.nombre, cursos: cursos, contactos: contactos }
```

33. ArangoSearch: SEARCH en Views

SEARCH se usa con Views de ArangoSearch, no directamente como FILTER común.

```
FOR doc IN vista_estudiantes
  SEARCH ANALYZER(doc.nombre IN TOKENS(@texto, "text_es"), "text_es")
  SORT BM25(doc) DESC
  LIMIT 10
  RETURN {
    nombre: doc.nombre,
    score: BM25(doc)
  }
```

Cuándo usarlo

Búsqueda de texto, ranking, scoring, analizadores lingüísticos y consultas tipo buscador.

34. WINDOW: agregados móviles

Ejemplo conceptual: promedio móvil de notas ordenadas por semestre.

```
FOR i IN inscripciones
  SORT i.semestre ASC
  WINDOW { preceding: 2, following: 0 }
  AGGREGATE promedio_movil = AVERAGE(i.nota)
  RETURN {
    semestre: i.semestre,
    nota: i.nota,
    promedio_movil: promedio_movil
  }
```

Uso típico

Series temporales, métricas acumuladas, promedios móviles y comparaciones locales.

35. Revisar planes de ejecución

En arangosh o desde la interfaz web se puede inspeccionar el plan:

```
db._explain(  
  FOR e IN estudiantes  
    FILTER e.carrera == "Bioinformatica"  
    SORT e.promedio DESC  
    RETURN e.nombre  
)
```

Busque señales como:

- EnumerateCollectionNode: posible full scan.
- Uso o no uso de índices.
- Sorts costosos.
- Cantidad estimada de documentos procesados.

36. Índices: se crean fuera de AQL

Ejemplo en arangosh:

```
db.estudiantes.ensureIndex({  
  type: "persistent",  
  fields: ["carrera"]  
})
```

// Luego la consulta AQL puede aprovecharlo:

```
FOR e IN estudiantes  
  FILTER e.carrera == "Bioinformatica"  
  RETURN e.nombre
```

Regla práctica

Indexar atributos usados frecuentemente en FILTER, SORT o patrones de búsqueda selectivos.

Patrones que escalan mejor

- Proyectar solo los atributos necesarios.
- Filtrar lo antes posible en la consulta lógica.
- Crear índices para filtros frecuentes y de alta selectividad.
- Evitar nested loops innecesarios en colecciones grandes.
- Usar edge collections y traversals cuando la relación es naturalmente un grafo.
- Usar `COLLECT AGGREGATE` cuando no se necesitan todos los documentos del grupo.
- Revisar `EXPLAIN` y `PROFILE` antes de optimizar a ciegas.

Errores comunes

- Usar AQL para crear colecciones o índices: eso no corresponde a AQL.
- Hacer RETURN doc en colecciones grandes sin necesidad.
- Usar LIMIT antes de filtrar u ordenar sin querer.
- Olvidar que las subconsultas devuelven arreglos.
- Confundir FILTER con PRUNE en traversals.
- Ejecutar UPDATE o REMOVE masivos sin probar primero el filtro.
- Concatenar parámetros como texto en vez de usar bind parameters.

Ejercicios simples

- 1 Listar los nombres de todos los estudiantes activos.
- 2 Listar estudiantes de Bioinformática ordenados por promedio descendente.
- 3 Devolver solo nombre, carrera y número de intereses de cada estudiante.
- 4 Buscar estudiantes cuyo arreglo `intereses` contenga "grafos".
- 5 Pagar estudiantes alfabéticamente, 2 por página.

Ejercicios intermedios

- 1 Contar estudiantes por carrera con `COLLECT`.
- 2 Calcular promedio de notas por curso.
- 3 Para cada estudiante, devolver una lista anidada de cursos inscritos.
- 4 Listar cursos con al menos 2 estudiantes inscritos.
- 5 Usar `UPSERT` para registrar una visita por estudiante.

Ejercicios avanzados

- ➊ Desde estudiantes/e1, listar contactos a distancia 1 y 2.
- ➋ Recomendar contactos de segundo grado excluyendo contactos directos.
- ➌ Listar todos los prerrequisitos de cursos/mineria usando traversal.
- ➍ Encontrar el camino más corto entre dos estudiantes.
- ➎ Comparar dos planes de ejecución: con y sin índice sobre carrera.

- AQL permite consultar documentos y grafos con un mismo lenguaje.
- La base de una consulta es `FOR + FILTER + RETURN`.
- `COLLECT` permite agrupar y agregar datos.
- Las subconsultas permiten resultados jerárquicos y consultas más expresivas.
- Las edge collections habilitan recorridos de grafos con `OUTBOUND`, `INBOUND` y `ANY`.
- La optimización requiere medir: `EXPLAIN`, `PROFILE` e índices adecuados.

Referencias principales

- ArangoDB Documentation. *AQL Documentation*. <https://docs.arango.ai/arangodb/stable/aql/>
- ArangoDB Documentation. *AQL Data Queries*.
<https://docs.arango.ai/arangodb/3.12/aql/data-queries/>
- ArangoDB Documentation. *High-level AQL operations*.
<https://docs.arango.ai/arangodb/3.12/aql/high-level-operations/>
- ArangoDB Documentation. *COLLECT operation in AQL*.
<https://docs.arango.ai/arangodb/3.12/aql/high-level-operations/collect/>
- ArangoDB Documentation. *Combining queries with subqueries in AQL*.
<https://docs.arango.ai/arangodb/stable/aql/fundamentals/subqueries/>
- ArangoDB Documentation. *Graph queries in AQL*.
<https://docs.arango.ai/arangodb/stable/aql/graph-queries/>
- ArangoDB Documentation. *AQL graph traversals explained*.
<https://docs.arango.ai/arangodb/stable/aql/graph-queries/traversals-explained/>